

Autor:
Ana Isabel Vegas



end2end dedication

CURSO

Java TDD Patrones y Buenas Prácticas



www.ibertech.es



end2end dedication

Tema 1 Introducción a Test Driven Development

Tema 2 Creando Test Complejos

Tema 3 Mocking con Mockito

Tema 4 Spring TDD

Tema 1

Introducción a Test Driven Development

Test Driven Development (TDD)

- Desarrollo guiado por pruebas, o Test Driven Development (TDD) es una práctica de programación que involucra otras dos prácticas:
 - Escribir las pruebas primero (Test First Development)
 - Refactorización (Refactoring).
- Para escribir las pruebas generalmente se utiliza la Prueba Unitaria

Test Driven Development (TDD)

- TDD (Test-Driven Development) es una metodología de desarrollo de software que consiste en escribir primero las pruebas unitarias antes de implementar el código de la funcionalidad. Este enfoque sigue un ciclo iterativo conocido como "rojo-verde-refactorizar":
 - **Fase roja:** Se escribe una prueba que inicialmente falla.
 - **Fase verde:** Se implementa el código necesario para que la prueba pase.
 - **Fase de refactorización:** Se optimiza el código sin cambiar su comportamiento.
- El objetivo principal de TDD es crear un código más robusto, modular y de mayor calidad. Al escribir las pruebas primero, los desarrolladores se centran en una única característica a la vez, lo que mejora la arquitectura de la solución y facilita la detección temprana de errores.

Ventajas TDD

- **Mejora la calidad del código:** Al escribir pruebas antes de implementar la funcionalidad, se asegura que cada línea de código esté justificada y cumpla con los requisitos establecidos.
- **Código más simple y menos redundante:** El ciclo constante de refactorización promueve la eliminación de código innecesario y la reutilización de funciones existentes.
- **Aumenta la productividad:** Se reduce el tiempo dedicado a la depuración, ya que los errores se detectan y corrigen tempranamente en el proceso de desarrollo.
- **Facilita la escalabilidad:** El enfoque modular del TDD permite crear código más fácil de mantener y expandir a medida que el proyecto crece.
- **Minimiza errores:** La detección temprana de problemas reduce significativamente el número de errores que llegan a producción.

Ventajas TDD

- **Mejora el diseño del software:** El proceso iterativo de TDD fomenta un diseño emergente y más efectivo de la arquitectura del software.
- **Proporciona documentación actualizada:** Los propios tests sirven como documentación ejecutable del comportamiento esperado del código.
- **Facilita la refactorización:** Permite mejorar el código existente sin temor a introducir nuevos errores, ya que las pruebas garantizan que la funcionalidad se mantiene.
- **Mejora la comunicación en el equipo:** Los problemas se abordan en períodos cortos de tiempo, fomentando la colaboración entre los miembros del equipo.
- **Aumenta la confianza en el código:** La alta cobertura de pruebas que se logra con TDD proporciona mayor seguridad sobre la funcionalidad del código en diversos escenarios.

El algoritmo TDD

- Escribir la prueba. Para escribir la prueba, el desarrollador debe entender claramente las especificaciones y los requisitos. El diseño del documento deberá cubrir todos los escenarios de prueba y condición de Excepciones.
- Escribir el código haciendo que pase la prueba. Este paso fuerza al programador a tomar la perspectiva de un cliente considerando el código a través de sus interfaces. Ésta es la parte conducida por el diseño, del TDD. Como parte de la calibración de la prueba, el código debe fallar la prueba significativamente las primeras veces.
- Ejecutar las pruebas automatizadas. Si pasan, el programador puede garantizar que el código resuelve los casos de prueba escritos. Si hay fallos, el código no resolvió los casos de prueba.
- Refactorización y limpieza en el código. Después se vuelven a efectuar los casos de prueba y se observan los resultados.
- Repetición. Después se repetirá el ciclo y se comenzará a agregar las funcionalidades adicionales o a arreglar cualquier error.

Prueba Unitaria

- Una prueba unitaria es una forma de probar el correcto funcionamiento de un módulo de código. Esto sirve para asegurar que cada uno de los módulos funcione correctamente por separado.
- La idea es escribir casos de prueba para cada función no trivial o método en el módulo de forma que cada caso sea independiente del resto. Esto último es la esencia de una prueba unitaria: se prueba al componente de forma aislada a todos los demás.

Características de las pruebas unitarias

- Para que una prueba unitaria sea buena se deben cumplir los siguientes requisitos:
 - **Automatizable:** no debería requerirse una intervención manual. Esto es especialmente útil para Integración Continua.
 - **Completas:** deben cubrir la mayor cantidad de código.
 - **Repetibles o Reutilizables:** no se deben crear pruebas que sólo puedan ser ejecutadas una sola vez. También es útil para Integración Continua.
 - **Independientes:** la ejecución de una prueba no debe afectar a la ejecución de otra.
 - **Profesionales:** las pruebas deben ser consideradas igual que el código, con la misma profesionalidad, documentación, etc.

Ventajas en el uso de Pruebas Unitarias

- **Detección temprana de errores:** Identifican problemas en etapas iniciales del desarrollo.
- **Facilitan el mantenimiento:** Aseguran que los cambios en el código no rompan funcionalidades existentes.
- **Documentación viviente:** Sirven como una guía clara sobre cómo debería comportarse cada unidad.

JUnit

- JUnit es un conjunto de clases (framework) que permite realizar la ejecución de clases Java de manera controlada, para poder evaluar si el funcionamiento de cada uno de los métodos de la clase se comporta como se espera.
- En función de algún valor de entrada se evalúa el valor de retorno esperado
- Si la clase cumple con la especificación, entonces JUnit devolverá que el método de la clase pasó exitosamente la prueba
- En caso de que el valor esperado sea diferente al que regresó el método durante la ejecución, JUnit devolverá un fallo en el método correspondiente

Ejecución de JUnit

- Hasta Versión **JUnit3**:
 - Definir clases que hereden de la clase `junit.framework.TestCase` de Junit.
 - Implementar métodos `testXXX()` (sin parámetros), que serán los que hagan las pruebas necesarias.
 - Inicialización del test en el método `setUp()` para colocar dicho código que se ejecutará antes de realizar cada prueba.
 - Terminación del test en el método `tearDown()` en el caso de que queramos liberar algunos recursos después de cada prueba.
 - Clases `AssertXXX` para comprobación de los resultados: `AssertTrue`
 - Lanzamiento de conjunto de tests mediante método `suite()`:

Ejecución de JUnit

- Versión **JUnit4**:
 - Incluye anotaciones (Java 5 annotations) en lugar de utilizar herencia:
 - `@Test` sustituye a la herencia de `TestCase`.
 - `@Before` y `@After` como sustitutos de `setUp` y `tearDown`.
 - `@BeforeClass` y `@AfterClass`, metodos estáticos para inicializar y liberar recursos en el Test

Ejecución de JUnit

- Versión **JUnit5**:
 - Permite trabajar con todas las novedades incluidas en Java 8:
 - Programación funcional
 - Lambdas
 - Streams, ...etc.

Principales anotaciones en Junit 5

- **@Test:** Indica que un método es un caso de prueba.
- **@DisplayName:** Permite definir un nombre personalizado para una clase de prueba o un método de prueba, mejorando la legibilidad de los resultados.
- **@BeforeEach:** Indica que el método anotado se ejecutará antes de cada método de prueba (reemplaza a @Before de JUnit 4).
- **@AfterEach:** Indica que el método anotado se ejecutará después de cada método de prueba (reemplaza a @After de JUnit 4).
- **@BeforeAll:** Indica que el método anotado se ejecutará una vez antes de todos los métodos de prueba en la clase (reemplaza a @BeforeClass).
- **@AfterAll:** Indica que el método anotado se ejecutará una vez después de todos los métodos de prueba en la clase (reemplaza a @AfterClass).

Principales anotaciones en JUnit 5

- **@Nested**: Indica que la clase anotada es una clase de prueba anidada y no estática.
- **@Tag**: Declara etiquetas para filtrar pruebas.
- **@Disabled**: Desactiva temporalmente una prueba (reemplaza a @Ignore de JUnit 4).
- **@TestFactory**: Denota un método que es una fábrica de pruebas para pruebas dinámicas.
- **@ParameterizedTest**: Indica que un método es una prueba parametrizada.
- **@RepeatedTest**: Permite crear pruebas que se ejecutan varias veces de forma automática.

Principales métodos de Aserciones en JUnit 5

- En JUnit 5, los métodos de aserción son fundamentales para validar los resultados de las pruebas unitarias. Estos métodos están disponibles en la clase estática `org.junit.jupiter.api.Assertions`. A continuación, se describen los principales métodos de aserción:

1. `assertEquals(expected, actual):`

- Verifica que el valor esperado sea igual al valor actual.
- Soporta un tercer argumento opcional para un mensaje personalizado y un parámetro delta para comparaciones con valores decimales.

```
assertEquals(5, 2 + 3, "Los valores no son iguales");
```

2. `assertNotEquals(expected, actual):`

- Verifica que dos valores sean diferentes.

Principales métodos de Aserciones en Junit 5

3. `assertTrue(condition)` / `assertFalse(condition)`:

- Verifica que una condición sea verdadera o falsa respectivamente.

```
assertTrue(5 > 3, "La condición no es verdadera");
```

4. `assertNull(object)` / `assertNotNull(object)`:

- Verifica que un objeto sea nulo o no nulo.

```
assertNotNull(myObject, "El objeto no debe ser nulo");
```

Principales métodos de Aserciones en Junit 5

5. `assertAll(heading, executable...)`:

- Agrupa múltiples aserciones y ejecuta todas, incluso si alguna falla.

```
assertAll("Validaciones",  
    () -> assertEquals(10, 5 + 5),  
    () -> assertNotNull("Texto")  
);
```

6. `assertThrows(expectedType, executable)`:

- Verifica que se lance una excepción específica durante la ejecución de un bloque de código.

```
assertThrows(IllegalArgumentException.class, () -> {  
    throw new IllegalArgumentException("Error");  
});
```

Principales métodos de Aserciones en Junit 5

7. **assertIterableEquals(expected, actual):**

- Compara dos objetos Iterable para verificar que sean iguales.

8. **assertArrayEquals(expectedArray, actualArray):**

- Compara dos arreglos para verificar que sean iguales.

9. **assertLinesMatch(expectedLines, actualLines):**

- Verifica que dos listas de cadenas coincidan línea por línea.

10. **fail(message):**

- Marca explícitamente una prueba como fallida con un mensaje personalizado.

Tema 2

Creando Test Complejos

Inicializadores

- Sirven para preparar y limpiar el entorno de test.

```
@BeforeEach
void setUp() {
    user = new User("test@mail.com");
}

@AfterEach
void tearDown() {
    // limpiar recursos si hace falta
}
```

Test y Cobertura

- La cobertura mide cuánto código ejecutan tus tests, pero:
 - Más cobertura $\neq$ mejores tests
- Herramientas típicas:
 - JaCoCo (muy usado en Spring)

Manejo de excepciones

```
@Test
void shouldThrowExceptionWhenUserNotFound() {
    when(repo.findById(1L)).thenReturn(Optional.empty());

    assertThrows(RuntimeException.class, () -> {
        service.getUser(1L);
    });
}
```

Anotaciones @Before y @After

- Úsalos cuando:
 -  Hay objetos comunes
 -  Hay configuración repetida
 -  NO para lógica compleja

Como construir test solidos

- Un buen patrón:
 - AAA Pattern (Arrange - Act - Assert)

```
@Test
void shouldCreateUser() {

    // Arrange
    when(repo.save(any())).thenReturn(new User("Juan"));

    // Act
    User result = service.create("Juan");

    // Assert
    assertEquals("Juan", result.getName());
}
```

El principio FIRST

- Reglas de oro de buenos tests:
 - Fast → rápidos
 - Independent → independientes
 - Repeatable → reproducibles
 - Self-validating → sin intervención manual
 - Timely → escritos a tiempo (TDD)

El principio BICEPS

- Menos conocido pero muy útil:
 - Boundary → prueba límites
 - Inverse → prueba lo contrario
 - Cross-check → valida con otra fuente
 - Error → fuerza errores
 - Performance → considera rendimiento
 - Stress → casos extremos

El principio CORRECT

- Para asegurar calidad en tests:
 - Conformance → cumple requisitos
 - Ordering → orden correcto
 - Range → valores dentro/fuera de rango
 - Reference → compara con referencia válida
 - Existence → null, vacíos, etc.
 - Cardinality → 0, 1, muchos
 - Time → timing, concurrencia

Test y Refactorings

- Clave en TDD:
 - 👉 Los tests te permiten refactorizar sin miedo
- Flujo:
 - Test pasa ✅
 - Refactorizas código
 - Test sigue pasando → todo OK

Tema 3

Mocking con Mockito

Mock Object

- Un Mock Object es un "objeto falso", un objeto que representa a otro y lo sustituye en funcionalidad. Este patrón es utilizado ampliamente en la Prueba Unitaria para asegurar un correcto aislamiento de la clase bajo test.
- Así, las dependencias que tenga nuestro objeto a testear pueden ser reemplazadas por mocks que funcionen como nosotros queremos. De lograr esto, podremos testear en forma aislada a nuestra clase, sin preocuparnos por sus dependencias (más aún, sin preocuparnos por si realmente funcionan estas dependencias).
- Usando Mock Objects podemos asegurar un "entorno perfecto y a medida", haciendo que este entorno responda como nosotros necesitamos. Luego, si el test de la clase falla, será por un problema en esta misma clase (y no en sus dependencias ya que, por hipótesis, el entorno era ideal).

Frameworks de Mocks

- Existen varios frameworks que ayudan a la creación de mocks:
 - EasyMock
 - Mockito
 - MockEjb
- Mockito es uno de los más conocidos.
- EasyMock es el usado por el equipo de Spring para testear su framework.

Mockito

- Mockito es una librería Java para la creación de Mock Object muy usados para el testeo unitario
- Mockito fue creado con el objetivo de simplificar y solucionar algunos de los temas antes mencionados.
- EasyMock y Mockito puede hacer exactamente las mismas cosas, pero Mockito tiene un API más natural y práctico de usar.

Principales anotaciones de Mockito

- Mockito es un framework de pruebas unitarias en Java que permite simular objetos y comportamientos. Sus anotaciones simplifican la creación, configuración e inyección de mocks en los tests. A continuación, se describen las principales anotaciones:
- **@Mock:**
 - Crea un objeto simulado (mock) de una clase o interfaz.
 - Se utiliza para reemplazar dependencias reales con simulaciones en las pruebas.

```
@Mock  
private UsuarioRepositorio usuarioRepositorio;
```

Principales anotaciones de Mockito

- **@InjectMock:**

- Crea una instancia del objeto que se está probando e inyecta automáticamente los mocks creados con @Mock o @Spy en sus dependencias.

```
@InjectMocks  
private UsuarioServicio servicio;
```

- **@Spy:**

- Envuelve un objeto real para espiar su comportamiento, permitiendo llamar a métodos reales y simular otros.

```
@Spy  
private List<String> lista = new ArrayList<>();
```

Principales anotaciones de Mockito

- **@Captor:**
 - Crea un captor para capturar argumentos pasados a métodos simulados.

```
@Captor  
private ArgumentCaptor<String> captor;
```

- **@MockBean:**
 - Reemplaza un bean real por un mock dentro del contexto de Spring.

```
@MockBean  
private Repositorio repositorioMock;
```

Principales anotaciones de Mockito

- **@ExtendWith(MockitoExtension.class):**
 - Habilita el uso de anotaciones de Mockito en JUnit 5, inicializando automáticamente los mocks y spies.

```
@ExtendWith(MockitoExtension.class)
class MiTest {
    @Mock
    private MiDependencia dependencia;
}
```

Principales métodos de Mockito

- Mockito ofrece una variedad de métodos para crear y manejar objetos simulados (mocks) en pruebas unitarias. A continuación, se describen los métodos más importantes:
- **mock(Class<T> classToMock):**
 - Crea un mock de una clase o interfaz específica.

```
List<String> mockList = Mockito.mock(List.class);
```

- **when(T methodCall):**
 - Define el comportamiento esperado de un método simulado.

```
Mockito.when(mockList.size()).thenReturn(5);
```

Principales métodos de Mockito

- **thenReturn(value):**

- Especifica el valor que debe devolverse cuando se llama al método simulado.

```
Mockito.when(mockList.get(0)).thenReturn("Hola");
```

- **verify(mock).method():**

- Verifica si un método específico fue llamado en el mock, con los argumentos esperados.

```
Mockito.verify(mockList).add("Elemento");
```

- **doReturn(value).when(mock).method():**

- Similar a when(), pero se utiliza cuando el método simulado no puede ser interceptado directamente (por ejemplo, métodos void).

```
Mockito.doReturn("Hola").when(mockList).get(0);
```

Principales métodos de Mockito

- **doNothing().when(mock).method():**

- Define que no se haga nada cuando se llame a un método void.

```
Mockito.doNothing().when(mockList).clear();
```

- **doThrow(exception).when(mock).method():**

- Configura un mock para lanzar una excepción específica cuando se llama a un método.

```
Mockito.doThrow(new RuntimeException()).when(mockList).clear();
```

- **assertThatThrownBy(() -> ...) (junto con assertThrows):**

- Verifica que un método lance una excepción esperada.

Principales métodos de Mockito

- **reset(mock):**
 - Restablece un mock para eliminar configuraciones previas.

```
Mockito.reset(mockList);
```

- **spy(T object):**
 - Crea un objeto espía que permite llamar a métodos reales y simular otros.

```
List<String> spyList = Mockito.spy(new ArrayList<>());
```

- **times(n) / atLeast(n) / never() (usados con verify):**
 - Verifican la cantidad de veces que un método fue invocado.

```
Mockito.verify(mockList, Mockito.times(2)).add("Elemento");
```

Tema 4

Spring TDD

Spring TDD

- Spring + TDD no es una tecnología nueva como tal, sino la combinación de dos cosas:
 - Spring Framework (o Spring Boot): para construir aplicaciones Java (APIs, servicios, etc.).
 - Test-Driven Development (TDD): una forma de programar guiada por pruebas.
- Tipos de tests en Spring TDD
 - Unit tests → servicios (con Mockito)
 - Integration tests → base de datos, contexto Spring
 - Controller tests → con MockMvc

Testing y dependencias con Spring

- El starter de test de Spring Boot ya te trae todo lo necesario para TDD:
 - JUnit → framework de tests
 - Mockito → mocks y spies
 - Spring Test → utilidades para testear Spring
 - AssertJ / Hamcrest → assertions más legibles
 - MockMvc → testear controllers sin levantar servidor

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-test</artifactId>  
  <scope>test</scope>  
</dependency>
```

Testing y JPA

```
<!-- JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Base de datos en memoria para tests -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Testing y JPA

```
@DataJpaTest
class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldFindUserByEmail() {
        User user = new User("test@mail.com");
        userRepository.save(user);

        Optional<User> result = userRepository.findByEmail("test@mail.com");

        assertTrue(result.isPresent());
    }
}
```

¿Qué hace @DataJpaTest?

- Levanta solo la capa JPA (rápido)
- Configura una DB en memoria (H2)
- Hace rollback después de cada test
- No levanta todo Spring (más eficiente)

Testing y JPA

- Lo que NO debes hacer
 - Mockear repositorios (@Mock)
 - Testear queries sin base de datos
- Repository tests (los más importantes)
 - Queries derivadas (findBy...)
 - Queries con @Query
 - Relaciones (@OneToMany, etc.)

Mocks y Stubs con Spring y Mockito

- Stub → controla lo que devuelve una dependencia
- Mock → verifica cómo se usa esa dependencia
- En la práctica: Mockito te da ambas cosas con el mismo objeto.

- **Stub** (definir comportamiento)

- Un stub le dice al mock qué debe devolver:

```
when(userRepository.findById(1L))  
    .thenReturn(Optional.of(new User("Juan")));
```

- **Mock** (verificar interacción)

- Después ejecutas tu código y verificas:

```
verify(userRepository).findById(1L);
```

Mocks y Stubs con Spring y Mockito

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void shouldReturnUserById() {
        // STUB
        when(userRepository.findById(1L))
            .thenReturn(Optional.of(new User("Juan")));

        // EXECUTE
        User result = userService.getUser(1L);

        // ASSERT
        assertEquals("Juan", result.getName());

        // MOCK (verify)
        verify(userRepository).findById(1L);
    }
}
```

Test de integración Repositorios y Servicios

```
@SpringBootTest
@Transactional
class UserServiceIntegrationTest {

    @Autowired
    private UserService userService;

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldCreateAndFindUser() {

        // create
        userService.create("test@mail.com");

        // verify via service
        User user = userService.findByEmail("test@mail.com");

        assertEquals("test@mail.com", user.getEmail());
    }
}
```

@SpringBootTest → levanta TODO el contexto
@Transactional → rollback automático tras el test
DB en memoria (H2 normalmente)
Sin mocks → todo real

Mocking Spring MVC

- En un test MVC quieres validar:
 - Rutas (@GetMapping, @PostMapping)
 - JSON de entrada/salida
 - Códigos HTTP
 - Validaciones

```
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;
}
```

@WebMvcTest → levanta solo la capa web

MockMvc → simula peticiones HTTP

@MockBean → sustituye el service real por un mock

Mocking Spring MVC

```
@Test
void shouldReturnUser() throws Exception {

    // STUB
    when(userService.getUser(1L))
        .thenReturn(new User("Juan"));

    // REQUEST + ASSERT
    mockMvc.perform(get("/users/1"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.name").value("Juan"));

    // VERIFY (opcional)
    verify(userService).getUser(1L);
}
```



end2end dedication

Muchas gracias por vuestra asistencia



www.ibertech.es